Component-and-connect patterns:

- Broker
- Model-view Controller
- Pipe and Filter
- Client-Server
- Peer-to-Peer
- Service-Oriented-Architecture
- **Publish-Subscribe**
- Shared-Data

Publish-Subscribe Pattern

- **Context:** There are a number of independent producers and consumers of data that must interact. The precise number and nature of the data producers and consumers are not predetermined or fixed, nor is the data that they share.

- **Problem:** How can we create integration mechanisms that support the ability to transmit messages among the producers and consumers so they are unaware of each other's identity, or potentially even their existence?

- **Solution:** In the publish-subscribe pattern, components interact via announced messages, or events. Components may subscribe to a set of events. Publisher components place events on the bus by announcing them; the connector then delivers those events to the subscriber components that have registered an interest in those events.

Publish-Subscribe Solution – 1

- Overview: Components publish and subscribe to events. When an event is announced by a component, the connector infrastructure dispatches the event to all registered subscribers.
- **Elements**:
  - *Any C&C component* with at least one publish or subscribe port.
  - *The publish-subscribe connector*, which will have *announce* and *listen* roles for components that wish to publish and subscribe to events.

- **Relations**: The *attachment* relation associates components with the publish-subscribe connector by prescribing which components announce events and which components are registered to receive events.

Publish-Subscribe Example

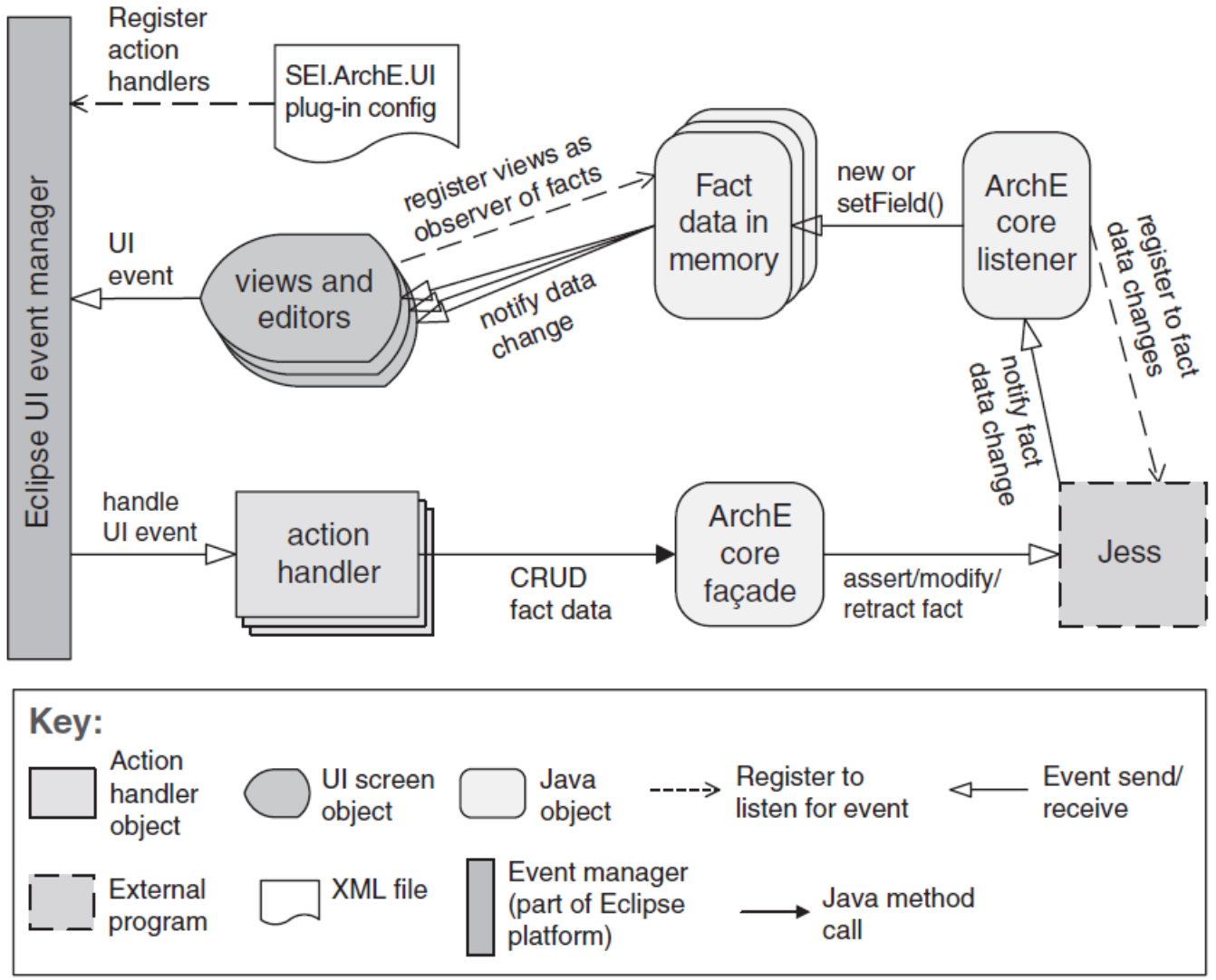

FIGURE 13.12 A typical publish-subscribe pattern realization

Publish-Subscribe Solution - 2

- **Constraints**: All components are connected to an event distributor that may be viewed as either a bus—connector—or a component. Publish ports are attached to announce roles and subscribe ports are attached to listen roles.

- **Weaknesses**:
  - Typically increases latency and has a negative effect on scalability and predictability of message delivery time.
  - Less control over ordering of messages, and delivery of messages is not guaranteed.

Publish-Subscribe Solution - 3

- **Advantages**
  - The publish-subscribe pattern is used to send events and messages to an unknown set of recipients. Because the set of event recipients is unknown to the event producer, the correctness of the producer cannot, in general, depend on those recipients. Thus, new recipients can be added without modification to the producers.
  - Having components be ignorant of each other's identity results in easy modification of the system (adding or removing producers and consumers of data)

Publish-Subscribe Solution - 4

- The publish-subscribe pattern can take several forms:
  - List-based publish-subscribe
  - Broadcast-based publish-subscribe
  - Content-based publish-subscribe

Publish-Subscribe Solution - 5

- The publish-subscribe pattern can take several forms:
  - List-based publish-subscribe:
  - Is a realization of the pattern where every publisher maintains a subscription list—a list of subscribers that have registered an interest in receiving the event.
  - This version of the pattern is less decoupled than others and hence:
    - it does not provide as much modifiability,
    - but it can be quite efficient in terms of runtime overhead.

Publish-Subscribe Solution - 6

- The publish-subscribe pattern can take several forms:
  - Broadcast-based publish-subscribe:
  - differs from list-based publish-subscribe in that publishers have less (or no) knowledge of the subscribers.
  - Publishers simply publish events, which are then broadcast. Subscribers (or in a distributed system, services that act on behalf of the subscribers) examine each event as it arrives and determine whether the published event is of interest.
  - This version has the potential to be very inefficient if there are lots of messages and most messages are not of interest to a particular subscriber.

Publish-Subscribe Solution - 7

- The publish-subscribe pattern can take several forms:
  - Content-based publish-subscribe is distinguished from the previous two variants, which are broadly categorized as "topic-based.". Topics are predefined events, or messages, and a component subscribes to all events within the topic.
    - Content, on the other hand, is much more general.
    - Each event is associated with a set of attributes and is delivered to a subscriber only if those attributes match subscriber-defined patterns.

Usage Examples on Publish-Subscribe

- Graphical user interfaces, in which a user's low-level input actions are treated as events that are routed to appropriate input handlers.
- MVC-based applications, in which view components are notified when the state of a model object changes
- Enterprise resource planning (ERP) systems, which integrate many components, each of which is only interested in a subset of system events
- Mailing lists, where a set of subscribers can register interest in specific topics
- Social networks, where "friends" are notified when changes occur to a person's website